Prediction-Aware Dynamic Task Scheduling on a Hybrid CPU/GPU Laptop: Architecture, Implementation and Measured Results

Lakshay S, Arkkadu Bharath Reddy
Reg. Nos. 26MAI0064, 26MAI0058 — Course: Computer Architecture and Organization
Guide: Dr. Yokesh Babu S, Associate Professor Grade 1. [Institution, department and e-mail to be filled by the authors]

***Abstract*—This draft describes the implementation of the project and analyses its results. The framework (12,600 lines of Python in the runtime, 9,700 in scripts, 7,300 in tests) executes the tasks of a dependency graph on pinned performance-core and efficiency-core worker processes and one CUDA worker (RTX 3050), logs every dispatch with its decision time, verifies every output against a CPU reference and compares scheduling policies in interleaved, seeded blocks with paired bootstrap intervals. The final benchmark compares *static* plans (HEFT, PEFT, HEFT with end-to-end terms) with *dynamic* dispatch (blind B4, a parameter-aware rule, and a predicted-rank, idle-worker dispatcher with and without end-to-end cost terms), all built on one ridge cost model, on six workloads and three hardware sets; the primary metric is the end-to-end makespan from the start of the run to the last completion. On the primary workload (24 included blocks, ten Holm-adjusted contrasts) the dynamic dispatcher is 17.8% faster than B4 on the CPU and 21.2% faster with a strict GPU set, and 5.2% (CPU) and 12.2% (strict GPU) faster than the static HEFT plan. The GPU helps the dispatcher (2.6% strict set, 7.5% extended set) but not blind B4. A hand rule that knows the cost parameter is within 1% of the dispatcher in every configuration, the policies tie when task cost does not vary, end-to-end cost terms change nothing (equivalent within ±2%), and a hybrid selector is not justified (maximum headroom 1.1%). The GPU cost estimate has 23–31% median error. Earlier exploratory sessions (hidden-cost workloads, an emulated remote worker) are reported as a first study; a Raspberry Pi has not been measured.**

***Index Terms*—*heterogeneous scheduling, DAG, runtime prediction, hybrid cores, CUDA, implementation, empirical evaluation, reproducibility*

I. Introduction

Reviews 1 and 2 surveyed the literature, patents and products and identified ten gaps, chiefly the lack of real-hardware evidence, overhead accounting, interval statistics, informed baselines and a like-for-like comparison of static plans with dynamic dispatch for DAG scheduling on heterogeneous resources. Review 3 reports the implementation built to close them and the results obtained with it. The contributions are: (1) a reproducible runtime and experiment framework for real task graphs on P/E cores, a CUDA GPU and an external worker; (2) a family of policies and estimators that includes the base idea B4, static HEFT and PEFT plans [1], [2] and a predicted-rank dispatcher; (3) a final benchmark of static against dynamic scheduling with one cost model, end-to-end timing and per-task GPU eligibility taken from measurements; (4) a measured map of when prediction helps, including the cases where it does not; (5) a diagnosis of scheduling overhead; and (6) a prepared path for a Raspberry Pi worker, validated only by emulation.

II. System Architecture

A. Overview

Fig. 1 shows the architecture. A workflow description (a Pegasus YAML file for Montage, or an in-memory image-chain generator) is converted into a DAG of tasks with declared inputs, outputs and eligible pools. The *coordinator* owns the ready queue, calls the scheduler policy whenever tasks become ready or workers become idle, and dispatches each task to a worker. Workers are separate processes pinned to one logical processor each (6 P workers or 2 P workers plus 7 E workers on the test machine); the coordinator runs on a reserved E core. A CUDA worker runs the GPU variants of three image stages, and a *remote worker* is a proxy process on the laptop plus a node process that serves tasks over TCP, on a local core behind a delay shim (emulation) or on another machine (real).

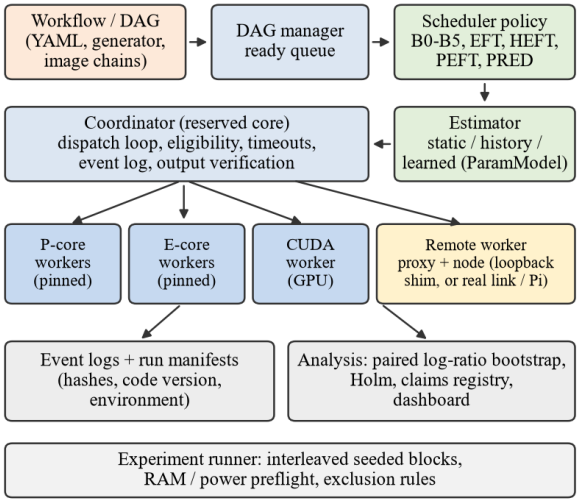


Fig. 1. Architecture of the implemented framework.

B. Runtime components

- **DAG and ready queue.** Tasks become ready when all parents completed (dependency counters, as in the base patent's in-degree gating); the ready queue is FIFO and policies may reorder it per dispatch pass.
- **Eligibility.** Every policy honours per-task eligible pools through one helper; the coordinator raises an error on an ineligible or non-idle placement (zero such errors in all reported sessions).

- **External-command tasks.** Montage jobs run as processes without a shell, with timeouts, process-tree kill, declared-output checks and hashing of inputs and outputs.
- **Failure semantics.** Per-task terminal status; dependents of a failed task are marked not-run; runs restart from a clean directory; failed or timed-out runs are kept and flagged.
- **Event log.** One JSON line per event (ready, dispatched, submitted, completed, failed); each dispatch carries the decision time and, for EFT-type policies, the candidate costs.
- **Remote path.** Frame-size caps, a token-based mutual HMAC handshake for non-loopback hosts, real artefact payloads, proxy-side timestamps for a foreign clock, and (added during this work) a persistent node and an adapter that connects to an already running node.

C. Estimators and policies

Estimators implement one interface and can be swapped without touching the scheduler: a constant per type and pool (E0), a solo-profile lookup (E1), an analytic size model (E2), a moving history (E3), a ridge model on occupancy context (E4), CatBoost and a small MLP, and an offline replay oracle (E5, reference only). The proposed *ParamModel* fits, per (stage, pool), ridge regression on the numeric task parameters and the size that vary in the logs and on their pairwise products; it names no stage or parameter and refuses a stage or pool it has not seen. Policies include B0–B5, random, EFT with each estimator, a rank-ordered EFT, HEFT, PEFT, informed hand rules, and *PredictedRankGreedy* (the proposed dispatcher): order ready tasks by predicted upward rank, bind only to an idle eligible worker, choose the one with the smallest predicted time.

D. Components added for the final benchmark

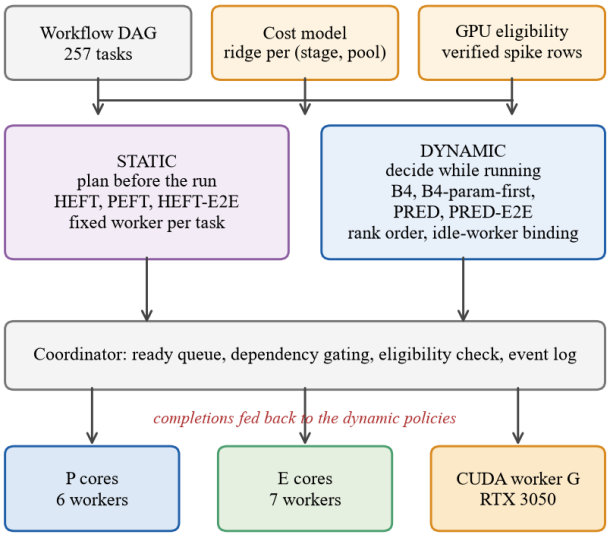


Fig. 2. Final framework: one cost model and one GPU-eligibility table feed both the static planners and the dynamic dispatchers; the coordinator, event log and worker pools are shared.

- **End-to-end timing (amendment F1).** The end-to-end makespan runs from the start of `run()` to the last completion; initialisation and policy set-up are recorded separately, so the time a static plan needs to be computed is counted. The older makespan (first dispatch to last completion) is kept.
- **Static baselines.** HEFT_param_model, PEFT_param_model and HEFT_E2E give every task a fixed worker before the run; an oracle plan from class-median costs is a reference only. Planned and actual makespan are logged.
- **Dynamic policies.** B4_param_first sorts the ready queue by the cost parameter. PRED_RANK_GREEDY orders by predicted upward rank and binds only to an idle worker with the least predicted kernel time. PRED_RANK_GREEDY_E2E minimises predicted occupancy, i.e. transfer plus kernel plus set-up plus inter-process time.
- **No double counting.** The CUDA stage time already contains the host-to-device copy, the kernel and the device-to-host copy (checked in the stage code); the end-to-end variant adds only artifact I/O and inter-process overhead.
- **GPU eligibility.** `GpuRegion` makes a stage eligible for the CUDA worker only where a spike record shows a verified speed-up; a dropped row overrides a pass. Configurations: *cpu*, *gpu* (strict: the original record) and *ext* (strict plus denoise at verified iteration counts). No configuration is GPU-only.

- **Separate data.** The cost model and the end-to-end terms are fitted on calibration sessions and checked on hold-out sessions; evaluation sessions come afterwards with new chain orders; the protocol and design hashes were logged before the first evaluation session.

E. Dispatch pass of the coordinator

Whenever a task becomes ready or a worker becomes idle the coordinator runs one *dispatch pass*. The pass is the same for all policies; only the two policy hooks differ:

1) take the ready tasks from the FIFO queue and call the policy's *order_ready* hook (default: unchanged; hand rules sort by a parameter; the proposed policy sorts by predicted upward rank);
2) for each task in the returned order, call *select_worker* (the policy returns a worker or nothing); a late-binding policy may only return an idle eligible worker;
3) check eligibility and idleness (a violation raises a contract error), bind the task, log the dispatch with the elapsed decision time and the policy's decision record;
4) stop the pass when no worker is idle (late-binding policies) or all ready tasks were visited; on completion the coordinator logs the result, releases dependents and calls the policy's feedback hook.

The coordinator accumulates the time spent in steps 1 and 2 as its *scheduling time*; the time of each successful call is attached to its dispatch as the *decision time*. Section IV-I shows why the two differ.

TABLE I

EVENT LOG AND RUN MANIFEST CONTENT

Record	Fields (selection)
task_ready, task_dispatched	task id, worker, pool, queue wait, worker load, decision time, decision record (candidate wait/transfer/exec/finish)
task_submitted, task_completed	kernel time, start/end timestamps, transfer time, bytes moved, output size and SHA-256
worker_ready, run_started, run_finished	pid, requested and actual affinity, capabilities; makespan, status, tasks per pool
run manifest	condition, policy, seed, instance, warm-up flag, status, error, output verdict and hashes, event-log SHA-256, git commit and dirty flag, source SHA-256, design SHA-256
session record	design hash, machine id, environment hash, blocks completed, preflight log, final status (complete, aborted, crashed)

F. Experiment framework and dashboard

A design file lists conditions, seed, block count, warm-up rule and thresholds; the runner derives a reproducible block plan, runs every condition once per block in random order, checks free memory and mains power before every block, writes a manifest per run (condition, seed, code hash, environment hash, output hashes, status) and makes event logs read-only. The analysis refuses to pool sessions with different code, environment, label or design, excludes runs by rules fixed in the design, and computes paired effects with bootstrap intervals, Holm-adjusted *p*-values, minimum detectable effects and a sensitivity pass. A claims registry with 50 entries is re-read against the stored tables by a test. A local web dashboard shows system state, a replay of runs and, on its last tab, any experiment session side by side (Fig. 3). Two later versions live in separate packages and leave the original unchanged: v2 shows the local GPU with live load, memory, temperature and power, and the final results with hover tooltips; v3 adds a tab that runs one live block of the final benchmark with any final policy on the CPU or CPU+GPU and shows the tasks per pool and the output check.

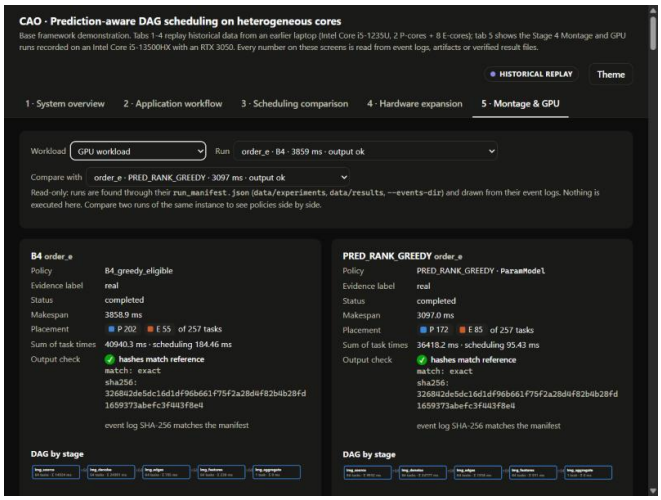


Fig. 3. Dashboard, tab "Montage & GPU": one B4 run (3858.9 ms) and one run of the proposed dispatcher (3097.0 ms) of the same chain order, with placement, scheduling time and output-hash check.

G. Code and test inventory

TABLE II

IMPLEMENTATION INVENTORY (MEASURED BY COUNTING LINES IN THE REPOSITORY)

Part	Files	Python lines
Runtime package (src/caosched)	107	12,578
core (coordinator, pools, DAG, tasks)		2,026
estimators; schedulers; analysis; experiment		1,197; 845; 1,037; 1,126
revival; study2 (earlier policies); final (static baselines, E2E dispatcher, GPU region)		1,327; 680; 216
dashboard (original; v2; v3); remote; dataset		774; 202; 266; 571; 692
Scripts (experiments, analysis, profiling, runbooks)	89	9,709
Tests	52	7,263 (521 tests passed)
Version control	71 commits	branch revival/resource-state-aware; final-study files not committed

III. Methodology

A. Platform and workloads

TABLE III

WORKLOADS

Workload	Tasks	Purpose
Montage mosaic (real, native Windows build, 9 DSS2 images)	59	real scientific DAG; nine 160 s reprojections dominate
Uniform image-chain batch (+ CUDA worker)	257	no hidden cost; no-harm control
Hidden-cost batch: H of 64 chains with 120 vs 2 denoise iterations	257	main study; H = 4, 8, 16, 40
Dose batch: heavy chains of 6, 15, 40, 120 iterations	257	cost-ratio sweep
Two-factor batch (size x iterations)	about 200	learned-model stress test
Image pipeline with remote stages (tile 512, 16 branches)	68	emulated remote worker; Pi path

TABLE IV

WORKLOADS OF THE FINAL BENCHMARK (IMAGE PIPELINE OF 257 TASKS, 512 PX; EVALUATION RUNS PER SESSION, INCLUDED RUNS IN BRACKETS)

Workload	Heavy chains of 64 / iterations	Runs
h16_l120 (primary)	16 heavy, 120 vs 2 iterations (ratio 60)	600 (576)
uniform	none (no spread in task cost); cpu and gpu only	208 (192)
h04_l120	4 heavy, 120 iterations	312 (288)
h40_l120	40 heavy, 120 iterations	312 (288)
h16_l40	16 heavy, 40 iterations (ratio 20)	312 (288)
h16_l15	16 heavy, 15 iterations (ratio 7.5)	312 (288)

The machine is a laptop with an Intel Core i5-13500HX (6 P and 8 E cores), an NVIDIA RTX 3050 6 GB (CUDA 12.8 build of PyTorch 2.11), 15.7 GB RAM and native Windows 11. Each session records an environment hash; results from different machines are never pooled. Montage was built natively with a MinGW-w64 toolchain after ten documented workarounds; three sequential reference runs gave identical mosaic hashes (maximum absolute difference 0.0), correct WCS headers and 147 dependency edges equal to the generator's own.

TABLE V

MONTAGE REFERENCE RUNS: MEDIAN WALL TIME PER TASK TYPE (THREE IDENTICAL SEQUENTIAL RUNS, 59 TASKS EACH)

Task type	n	Median (s)	Range (s)
mProject (reprojection)	27	160.57	141.02–169.89
mDiffFit (overlap fit)	108	0.251	0.041–1.036
mBackground	27	0.336	0.303–0.610
mConcatFit / mBgModel / mImgtbl	3 each	0.207 / 0.379 / 0.094	–
mAdd / mViewer	3 each	0.870 / 4.157	–
Total sequential makespan	3 runs	1459.4, 1467.1, 1469.3	outputs identical (SHA-256)

B. Procedure and statistics

Final benchmark: eight policies (four dynamic, four static of which the oracle plan is a reference) in three hardware sets, one run of every condition per block in seeded random order, the first block of a session a warm-up. Before every block the free memory had to exceed 2.4 GB at the start of a session and 0.8 GB per block, and the laptop had to be on mains power; these thresholds were never lowered, and attempts that failed the gate were kept on disk and never pooled. The confirmatory family is the ten contrasts C1–C10 on the primary session under Holm correction with a ±2% equivalence margin; the other five sessions are exploratory and unadjusted.

The effect of a condition against a reference is the paired geometric-mean change of makespan within interleaved blocks, with a percentile bootstrap interval stratified by workload instance (10,000 resamples) and a Holm-adjusted sign-flip p-value over the confirmatory family. Registered sessions were pre-registered with amendments. Overnight sessions are *exploratory*: plan and design hash were written before each run, each design was run once, and the plan fixed the reading of the result in advance. Chain orders are shuffles of the 64 chains seeded by an integer; orders used for model fitting (coverage) are disjoint from evaluation orders.

IV. Results

A. Final benchmark: primary workload

Table VI gives the end-to-end makespan (geometric mean over 24 included blocks) of every policy in the three hardware sets; Fig. 4 adds the 95% intervals. In every set the three dynamic policies that use cost information (B4 param-first, PRED, PRED-E2E) are within about 1% of each other and clearly faster than blind B4; the static HEFT plan is 2% to 12% slower than PRED, and PEFT is the slowest static plan. All 576 included runs of the session had verified output.

TABLE VI

PRIMARY WORKLOAD H16_L120: END-TO-END MAKESPAN IN SECONDS (GEOMETRIC MEAN, 24 BLOCKS)

Policy	CPU only	Strict GPU	Extended GPU
B4 (blind, dynamic)	4.010	4.075	4.123
B4 param-first (rule, dynamic)	3.310	3.218	3.069
PRED (dynamic)	3.297	3.213	3.050
PRED-E2E (dynamic)	3.318	3.212	3.033
HEFT (static)	3.478	3.660	3.114
HEFT-E2E (static)	3.499	3.512	3.173
PEFT (static)	4.093	4.050	3.689

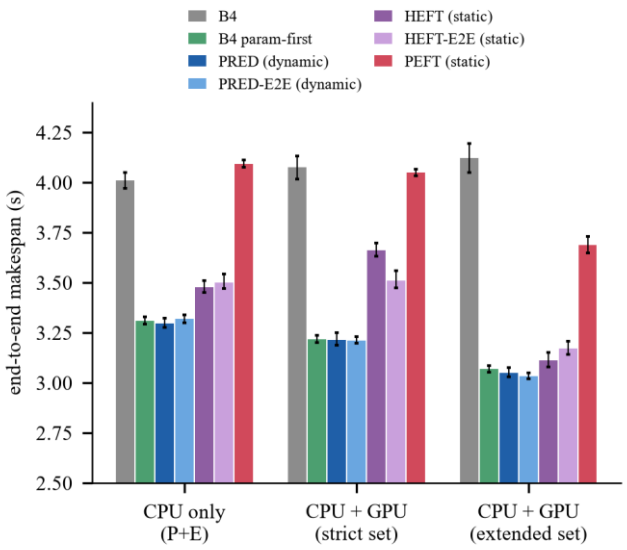


Fig. 4. End-to-end makespan of each policy in the three hardware sets on the primary workload (geometric mean, 95% interval, 24 blocks).

The ten confirmatory contrasts are in Table VII and Fig. 5. Predictions written before the data held for C1, C2, C7, C9 and C10. C3 and C4 had been predicted inconclusive or equivalent; the dynamic policy was about 5% faster, so that prediction was wrong. C5, C6 and C8 had no stated expectation.

TABLE VII

CONFIRMATORY CONTRASTS (HOLM FAMILY OF TEN; EFFECT ON END-TO-END MAKESPAN, NEGATIVE = FIRST POLICY FASTER)

Id	Contrast	Effect % [95% CI]	Holm p	Result
C1	PRED vs B4 (CPU)	−17.8 [−18.8, −16.7]	0.001	faster
C2	PRED-E2E vs PRED (CPU)	+0.6 [+0.1, +1.3]	0.141	equivalent (within ±2%)
C3	PRED vs HEFT (CPU)	−5.2 [−5.9, −4.6]	0.001	faster
C4	PRED-E2E vs HEFT-E2E (CPU)	−5.2 [−5.9, −4.5]	0.001	faster
C5	PRED vs HEFT (strict GPU)	−12.2 [−13.4, −11.1]	0.001	faster
C6	PRED-E2E vs HEFT-E2E (strict GPU)	−8.5 [−9.7, −7.6]	0.001	faster
C7	PRED-E2E vs PRED (strict GPU)	0.0 [−0.7, +0.5]	0.960	equivalent (within ±2%)
C8	PRED vs B4 (strict GPU)	−21.2 [−22.6, −19.7]	0.001	faster
C9	PRED, strict GPU vs CPU	−2.6 [−3.3, −1.7]	0.001	faster
C10	B4, strict GPU vs CPU	+1.6 [+0.2, +3.0]	0.141	inconclusive

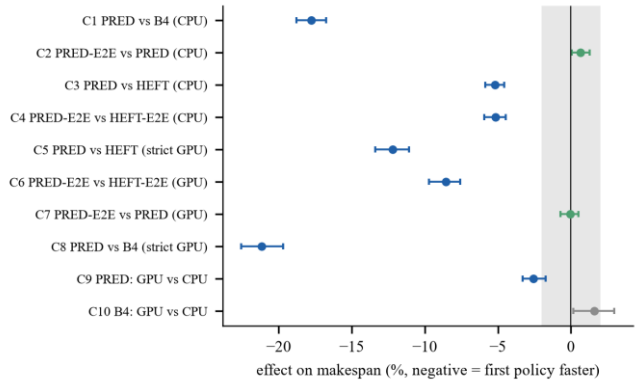


Fig. 5. Confirmatory contrasts C1–C10 with 95% intervals; the grey band is the ±2% equivalence margin.

B. Final benchmark: six workloads

The effect depends on how much task cost varies (Fig. 6, exploratory outside the primary session). Against static HEFT the dynamic policy gains 0.2% to 5.2% on the CPU and 3.7% to 17.1% with the strict GPU set; against blind B4 it gains almost 40% when only 4 of 64 chains are heavy, 2.8% to 4.8% when 40 are heavy or the cost ratio is 20, and nothing on the uniform workload (+0.0% [−2.5, +2.3] on the CPU; −0.0% [−1.3, +1.1] with the strict GPU set). With the lowest cost ratio (7.5) the dynamic policy is slightly slower than B4 with the strict GPU set (+1.1% [+0.3, +1.9]).

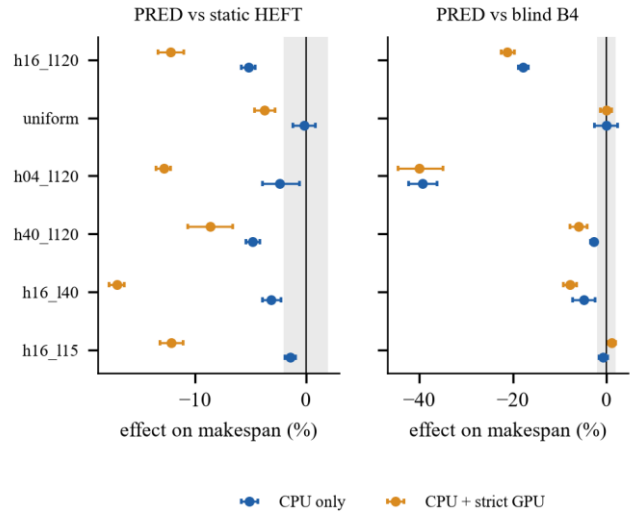


Fig. 6. PRED against static HEFT (left) and blind B4 (right) on the six workloads, CPU only (blue) and with the strict GPU set (orange); exploratory except the first row.

C. Final benchmark: GPU, cost-model error and end-to-end terms

The GPU helps only where it is used with a cost model: for PRED the strict set is 2.6% [−3.3, −1.7] and the extended set 7.5% [−7.8, −7.2] faster than the CPU only, while blind B4 is 1.6% *slower* with the strict set (C10, inconclusive) and 2.8% slower with the extended set. Under the model-based policies

the CUDA worker takes about 15% to 20% of the 257 tasks (Fig. 7). One CUDA worker is a single narrow resource.

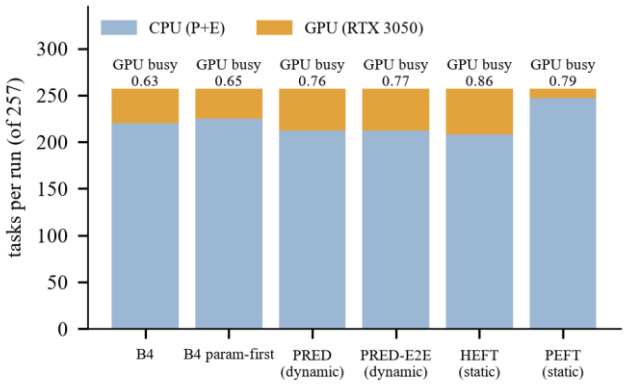


Fig. 7. Tasks run on the CPU and on the GPU per policy (extended set, primary workload) and the GPU busy fraction.

Model error. On hold-out sessions never used for fitting, CPU stages are predicted with a median error of 4% to 11%, GPU stages with 23% to 31% (Fig. 8). The GPU estimate is the weakest part of the model: a static plan commits to it in advance, a dynamic policy uses it only when a worker is idle, so part of the dynamic-versus-static gap on the GPU sets may reflect model error and not only the timing of decisions; this was not separated experimentally. **End-to-end terms.** Adding set-up and inter-process terms changed nothing: PRED-E2E against PRED is +0.6% [+0.1, +1.3] on the CPU (C2) and −0.0% [−0.7, +0.5] with the strict GPU set (C7), both equivalent within ±2%; on the uniform workload with the strict set it was +2.9% [+0.4, +5.9] slower than B4. **Timing convention.** The difference between the end-to-end and the old makespan is 12.5 ms (B4), 29.2 ms (PRED), 40.7 ms (HEFT) and 81.3 ms (PEFT) on a makespan of about 3 to 4 s, so no conclusion flips between the two conventions.

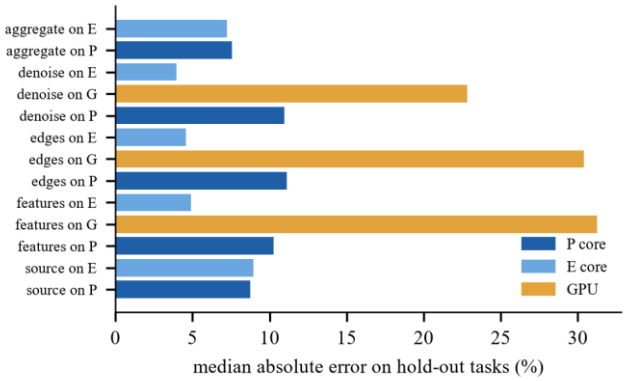


Fig. 8. Median absolute error of the cost model on hold-out tasks, by stage and pool.

D. Final benchmark: what limits the claim

- **A simple rule gets almost all of the gain.** PRED against B4 param-first (the queue sorted by the cost parameter) is −0.4% [−0.9, +0.1] on the CPU, −0.6% [−1.1, −0.0] with the extended set and −0.2% [−0.9, +0.7] with the strict set (exploratory): under 1% in every set.
- **A hybrid selector is not justified.** In 17 development cells the maximum headroom of a B4-or-PRED selector was 1.1%, the mean 0.08%, and no cell exceeded the 2% margin, so none was built.
- **Two sessions disagree.** The earlier overnight session found HEFT level with the dispatcher (+1.10% [−0.53, +2.81], old timing convention, other chain orders); the final session finds PRED about 5% faster. The timing convention does not explain it; the difference was not isolated and both results are reported.
- **Scope.** One machine, one workload family, 512 px, one CUDA worker; only the primary session is confirmatory; the remote worker was not rerun with the final policies.

E. Earlier study: Montage, a valid null result

On the 59-task mosaic with 6 P and 7 E workers (9 blocks, registered), predictive EFT variants and HEFT were 1.4% to 1.7% *slower* than static use of the P cores (intervals above 0), learned models 9% to 14% slower, and B4 47.9% slower (95% interval +37.5 to +54.6). The profile explains it: every Montage task prefers P cores by a factor of 2.3 to 3.6 and nine equal tasks dominate, so no placement can beat "use the P cores" and any use of E cores costs time.

F. Earlier study: hidden-cost workload

Registered sessions with two chain orders each gave the proposed dispatcher −20.39% [−21.2, −19.3] (30 blocks, 6P+7E) and −21.10% [−21.5, −20.7] (20 blocks, 2P+7E, model refitted by one command) relative to B4. To test generalisation, exploratory sessions used chain orders never used before (Table VIII, Fig. 9). The dispatcher was faster than B4 in all 12 and all 8 orders; the informed hand rule, which sorts chains by the heavy parameter without any model, gave the same gain, and the difference between dispatcher and rule had intervals inside ±2%.

TABLE VIII

UNSEEN CHAIN ORDERS (EXPLORATORY); EFFECTS RELATIVE TO B4 UNLESS NOTED (NEGATIVE = SHORTER)

Session	Blocks / orders	Dispatcher	Hand rule	P-only	Dispatcher vs rule
6P+7E, H=16	36 / 12	−19.32% [−20.0, −18.6]	−19.85%	+3.68%	+0.66% [−0.2, +1.5]
2P+7E, H=16	24 / 8	−20.45% [−21.2, −19.8]	−20.67%	+45.77%	+0.27% [−0.2, +0.8]
6P+7E, H=4	12 / 4	−27.65% [−34.0, −18.7]	−26.94%	−11.56%	−0.98% [−3.1, +1.2]
6P+7E, H=8	12 / 4	−26.76% [−27.9, −25.7]	−26.93%	−15.77%	+0.24% [−0.4, +1.0]
6P+7E, H=40	12 / 4	−3.81% [−5.1, −1.9]	−5.15%	+23.13%	+1.42% [−0.7, +4.5]

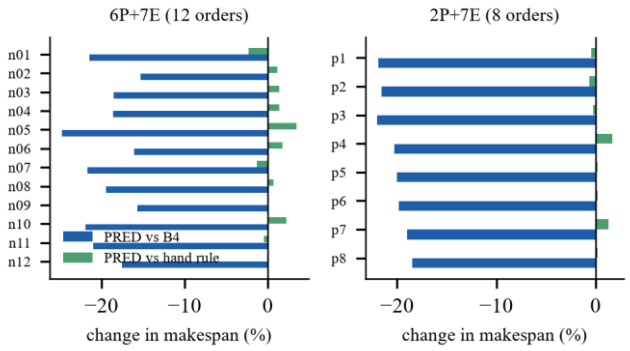


Fig. 9. Per-order change in makespan of the dispatcher against B4 (blue) and the hand rule (green).

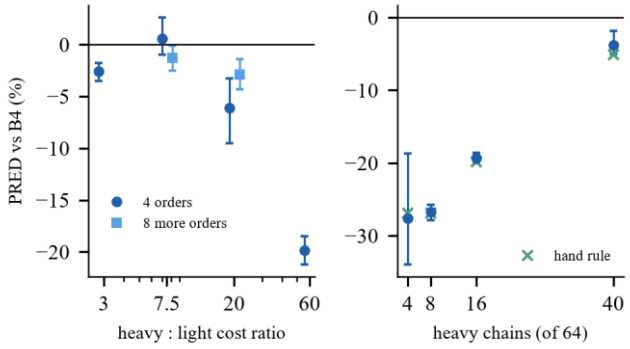


Fig. 10. Gain over B4 versus heavy:light cost ratio (left) and number of heavy chains (right).

Dose-response. Holding 16 heavy chains and raising their cost ratio from about 3 to 60 gave −2.6%, +0.6% (not significant), −6.1% and −19.9% in a first pass of four orders; a follow-up with eight further orders gave −1.3% (ratio 7.5) and −2.9% (ratio 20). The first-pass irregularity was order-specific: the gain is small, about 1% to 3%, up to a ratio of 20 and large only near 60. **Heavy-chain count.** With the ratio fixed the gain is largest when few chains are heavy and falls as they become the majority (Fig. 10), consistent with a tail effect in which blind dispatch starts a heavy chain late or on a slow core.

G. Registered results and model accuracy

TABLE IX

SELECTED REGISTERED CLAIMS (PRE-REGISTERED DESIGNS; EFFECT = PAIRED MAKESPAN CHANGE, NEGATIVE = SHORTER)

Claim	Workload	Comparison	Effect	Status
U1	uniform batch + GPU	predictive EFT vs static P-only	−27.56%	supported
U2	uniform batch + GPU	predictive EFT vs B4	+0.25%	equivalent (±2%)
H1	hidden cost (first run)	rank EFT with learned model vs B4	−13.78%	supported
G1, G3	hidden cost 6P+7E	dispatcher vs B4; table predictor vs dispatcher	−20.39%; +75.19%	supported
G4	hidden cost 6P+7E	dispatcher vs hand rule	−1.54% (rule faster)	not resolved
R1, R3	hidden cost 2P+7E	dispatcher vs B4; table predictor vs dispatcher	−21.10%; +57.65%	supported
R4	hidden cost 2P+7E	dispatcher vs hand rule	+0.38%	equivalent
N1	uniform batch	dispatcher vs B4	−0.45%	equivalent
C1, C2	two-factor	size-only rule, static P-only vs dispatcher	+17.44%; +17.13%	supported
C3	two-factor	dispatcher vs B4 (≥2% faster?)	+0.95% [−0.9, +2.8]	not supported
V1	contended mixed DAG	oracle gate (≥5% headroom)	B4 1.24% faster than oracle	gate failed

The runtime model is accurate where it matters. On a held-out two-factor session the predictions for the heavy classes were within 8% of the measured time (0.605 s predicted versus 0.565 s measured for 256-pixel chains with 300 iterations on a P core; 0.122 s versus 0.112 s for 1024-pixel chains) but the lightest class was poorly predicted (17.6 ms versus 4.1 ms on a P core), which does not matter for makespan because such tasks are short. Fitting uses a few coverage sessions on the same machine; a new pool layout is handled by the same command, as shown for the 2 P + 7 E configuration.

H. Established list schedulers and ablations

Driven by the same learned model on six further unseen orders (24 blocks), HEFT gave −18.08% and PEFT −2.58% relative to B4, rank-ordered EFT with early binding −13.21%, and the dispatcher −18.97%; HEFT versus the dispatcher was +1.10% [−0.53, +2.81], PEFT +20.23%, early-binding EFT +7.11%. The ordering and the late (idle-only) binding both contribute; the static plan of HEFT is competitive on this workload.

I. Scheduling overhead

Two overhead figures exist. *Per-dispatch decision time* (the time of the worker-selection call that bound a task) is 6.8 ms per run for B4 and 15.5 ms for the dispatcher (257 tasks, about 0.5% of makespan). *Coordinator scheduling time* (all calls of the policy, including those that returned nothing) is 0.197 s and 0.100 s. The difference was traced with an instrumented run: because a CUDA worker that no task can use is always idle, the dispatch loop never exits early, and 10,042 of 10,299 calls of B4's selection function returned nothing. A decision-preserving fix in a separate adapter reduced coordinator time to 8 ms (B4) and 24 ms (dispatcher) and moved the dispatcher's gain from −19.47% to −20.28% (0.8 percentage points) with all 78 runs verified; the fix is not applied to the source.

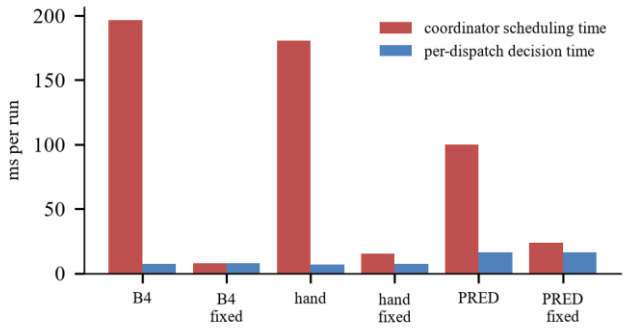


Fig. 11. Per-dispatch decision time and coordinator scheduling time per run, before and after the dispatch-loop fix.

J. GPU and remote execution

At task level the GPU is up to 15.1 times faster for the edge-detection stage at 1024 px and 5.2 times for feature extraction, with outputs equal within tolerance; the denoise stage gains only 1.4 times and not at small tiles. No workflow-level gain from CPU/GPU placement was established; in 20,046 stored B4 dispatches with the GPU worker, 93.6% had a single eligible idle worker. For remote execution, an emulated worker (a node on one local E core behind a loopback socket and a delay shim) was tested at two compute levels. At level 30 the predictive policy used the remote worker (1.7 tasks per run) with accurate predictions (predicted vs measured execution 0.1457 vs 0.1464 s and transfer 0.0937 vs 0.0943 s over 44 placements) yet was 3.78% [+2.27, +5.58] *slower* than without it (Fig. 12). Prediction accuracy did not become scheduling value in this regime. A loopback rehearsal of the complete Raspberry Pi command chain (profile, calibration with an external node, design generation, 25 runs, analysis) passed after a defect in the node (it served only one client) was fixed.

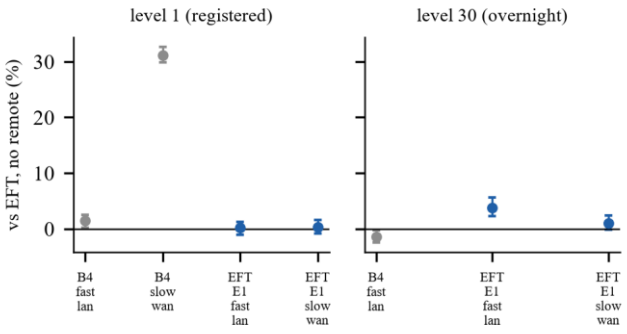


Fig. 12. Emulated remote worker: effect on EFT makespan at compute level 1 (registered) and level 30 (overnight); error bars are 95% intervals.

K. Summary of claims

TABLE X

CLAIM STATUS AFTER THE PROJECT (50 REGISTERED CLAIMS OF THE EARLIER STUDY PLUS THE FINAL BENCHMARK; SELECTION)

Status	Claim
Supported	dynamic PRED beats blind B4 by 17.8% (CPU) and 21.2% (strict GPU); beats a static HEFT plan from the same model by 5.2% and 12.2%; the GPU helps PRED (2.6% strict, 7.5% extended); end-to-end terms are equivalent; earlier study: rank-ordered dispatch beats B4 by 19–28% on cost-skewed workloads
Not established	advantage over an informed rule (within 1%); a hybrid selector; GPU benefit for blind B4; remote offloading benefit; Raspberry Pi results; any claim beyond this machine and workload family
Not supported / null	gain on Montage; gain on the two-factor workload; gain on the uniform workload; remote worker helping EFT at level 30

L. Sessions executed and integrity checks

TABLE XI

TIMED SESSIONS (ALL INCLUDED OUTPUTS VERIFIED; HASHES OF EVENT LOGS CHECKED)

Group	Sessions	Blocks	Runs	Label
Unseen orders 6P+7E / 2P+7E	2	37 / 25	148 / 100	real
Dose-response (ratio 3, 7.5, 20, 60) and follow-up (7.5, 20)	6	52 + 50	208 + 200	real
HEFT/PEFT/rank-EFT vs dispatcher	1	25	150	real
Heavy-chain count 4, 8, 40	3	39	156	real
Dispatch-loop A/B	1	13	78	real
Reproduction of the registered design	1	11	55	real
Final study: calibration (cal_l15, cal_l40, cal_l120, cal_uniform) and hold-out (hold_l40, hold_l120), completed retries	6	–	99 + 28	real
Final study: evaluation, six workloads	6	25, 13 × 5	600, 208, 312 × 4	real
Emulated remote pool, level 30	1	13	65	emulation
Loopback rehearsal of the Pi command chain	1 (+1 failed)	5	25 (+25 failed)	emulation

In every session of the table all runs finished with status ok and verified output, event-log hashes matched the manifests, the design hash matched, and a script re-derived each reported table from the raw logs (exit status 0 for every analysis). The only failed runs are the 24 of the first rehearsal attempt, kept as evidence of the node defect described below.

M. Defects found and corrected during the work

TABLE XII

DEFECTS, HOW THEY WERE FOUND AND WHAT WAS DONE

Defect	Found by	Action
Learned model applied to a different profile than it was fitted on	audit of earlier phases	profile binding recorded in the model; refusal on mismatch; tests
Worker-pool rebuild between conditions created cold workers (two-humped makespans)	pilot analysis	one pool for all conditions
Decision-context logging cost 3% of makespan in EFT	paired comparison	no-context variants; scheduling-cost guard (25%)
Remote node served one client then exited	loopback rehearsal (24 runs failed)	persistent mode and test
Calibration ignored the compute level; no external host option	code reading	options added; test
Idle CUDA worker defeats the dispatch loop's early exit	overhead diagnosis	quantified; fix in a separate adapter; source fix proposed
Analysis tool labels the primary family "confirmatory" by default	review	documented; overnight sessions stay exploratory by plan
Final-study stage 1 aborted when the laptop was unplugged, on retry-id collisions and on low free memory (antivirus, Docker, WSL)	run logs	mains wait and first-unused-id logic added; the 2.4 GB and 0.8 GB thresholds were never lowered; aborted attempts kept (_r2 … _r8), never pooled
Static plans were timed from the first dispatch, hiding their planning time	design review	end-to-end makespan (amendment F1); old metric kept

N. Raspberry Pi path (prepared, not measured)

The stage needs no further code change. On the Pi: a Python 3.11 environment with numpy, Pillow and psutil, the package installed without its heavy dependencies, and the node started with a token and the persistent option. On the laptop: a base profile at the experiment's compute level, link calibration against the Pi (kernel times, payload sizes, transfer time as a + b·bytes), a generated design with five conditions (static, B4 and EFT without remote; B4 and EFT with remote), 13 blocks per compute level, and the standard analysis with a decision rule fixed beforehand (the remote worker helps only if the interval of EFT with versus without it lies below zero). A runbook lists every command, the measurements to record on arrival (temperature, throttling flag, network throughput), and the rollback to the local demonstration.

V. ANALYSIS AND DISCUSSION

Mechanism. The dominant effect is ordering and informed placement: serving predicted-heavy work first removes the tail created by blind dispatch, and sending only verified stages to the GPU adds a smaller gain. That is why a rule that knows the cost parameter is within 1% of the dispatcher and why the effect depends on the spread of task cost. A dynamic policy also beats a static plan from the same model, by 5% on the CPU and 12% with a strict GPU set: the static plan cannot react to the actual completion order, and its GPU assignments inherit the 23–31% model error in advance. **Prediction accuracy is not enough:** remote predictions were accurate to under 1% yet gave no gain, in line with the finding of [3] that order can matter more than estimate accuracy. **Overhead matters but did not decide the result.** Scheduling time is 2% to 5% of these 3 to 9 second runs for late-binding policies; in the final benchmark policy set-up is 19 ms (PRED), 30 ms (HEFT) and 71 ms (PEFT).

Relation to the literature. The surveyed learned schedulers report 11% to 27% improvements in simulation [4], [5], [6], [7]. Our figures are measured on real hardware with intervals and an informed baseline, are of the same order on cost-skewed workloads, and show that most of the gain is available to a much simpler rule. We therefore do not claim superiority over learned DRL schedulers, which we did not run.

Threats to validity. The workload family is designed to have a hidden cost; one machine; runs with a dirty working tree and different source hashes (never pooled); overnight sessions exploratory; the first run after a reference computation is about 2% slower; HEFT and PEFT are static plans verified on the papers' worked examples; remote results are emulation. The Raspberry Pi stage will test whether a real remote worker that adds capacity changes the picture.

TABLE XIII

THIS IMPLEMENTATION COMPARED WITH THE CLOSEST PUBLISHED SYSTEMS

System	Evaluation	Hardware	Overhead reported	Intervals	Informed baseline
[4]	simulation	simulated cluster	inference time	no	no
[8]	trace simulation + testbed	3 Jetson boards	no	no	no
[9]	real	Zynq CPU+FPGA	yes	no	no
[10]	simulation	simulated heterogeneous cluster	prediction error	median errors	HEFT with true runtimes
[11]	real	CPU+GPU nodes	history models	not assessed	no
This work	real executions	P/E cores, RTX 3050, emulated remote	two metrics per run	paired bootstrap, Holm	yes (hand rules)

VI. Existing versus Proposed Implementation

TABLE XIV

DIFFERENTIATION BETWEEN THE EXISTING (BASE) IMPLEMENTATION AND THE PROPOSED ONE

Aspect	Existing (B4 / base patent idea)	Proposed
Order of ready tasks	first-come first-served	predicted upward rank
Worker choice	first idle eligible worker (P before E)	idle eligible worker with least predicted time (CPU or verified GPU stages)
Cost knowledge	none	model learned from logs; retrained by one command
Decision cost per run	7 ms (coordinator 0.20 s)	15 ms (coordinator 0.10 s); policy set-up 19 ms
Makespan, primary workload	reference	−17.8% (CPU), −21.2% (strict GPU) vs B4; −5.2% and −12.2% vs static HEFT
Makespan, nothing hidden	reference	+0.0% (equivalent within ±2%)
Evidence	—	real runs, end-to-end timing, paired intervals, ten Holm-adjusted contrasts, 521 tests

VII. Conclusion and Future Work

The implementation delivers a reproducible framework and a measured account of static against dynamic scheduling on a hybrid CPU+GPU laptop: dynamic dispatch with a cheap cost model is 2% to 12% faster than a static plan from the same model and about 18% to 21% faster than blind dispatch when task cost varies; it ties blind dispatch when it does not; a rule that knows the cost parameter gets almost the same result; and the GPU helps only with a cost model. Next: (1) a second workload family; (2) a GPU cost model with lower error and a test that separates model error from decision timing; (3) the remote worker with the final policies, then a Raspberry Pi over a real link with the prepared runbook; (4) apply and test the dispatch-loop fix in the source and commit the final study; and (5) obtain a professional prior-art search before any IPR filing.

References

[1] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[2] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014, doi: 10.1109/TPDS.2013.57.

[3] A. Ilyushkin and D. Epema, "The impact of task runtime estimate accuracy on scheduling workloads of workflows," in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGrid)*, 2018, pp. 331–341, doi: 10.1109/CCGRID.2018.00048.

[4] J. Luo, Y. Zhou, X. Li, M. Yuan, J. Zeng, and J. Yao, "Learning to optimize DAG scheduling in heterogeneous environment," in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, doi: 10.1109/MDM55031.2022.00040 (arXiv:2103.06980).

[5] J. Liu, P. Xie, W. Li, B. Tang, and J. Liu, "Dynamic task offloading scheme for edge computing via meta-reinforcement learning," *Comput. Mater. Contin.*, vol. 82, no. 2, 2025, doi: 10.32604/cmc.2024.058810.

[6] C. Liu, H. Wang, M. Zhao, J. Liu, X. Zhao, and P. Yuan, "Dependency-aware online task offloading based on deep reinforcement learning for IoV," *J. Cloud Comput.*, vol. 13, Art. no. 136, 2024, doi: 10.1186/s13677-024-00701-0.

[7] Q. Tan, W. Chen, and D. Liu, "Task scheduling for heterogeneous multicore processors based on deep reinforcement learning," *Int. J. Intell. Syst.*, vol. 2025, Art. no. 7562400, 2025, doi: 10.1155/int/7562400.

[8] X. Chen, J. Cao, Y. Sahni, S. Jiang, and Z. Liang, "Dynamic task offloading in edge computing based on dependency-aware reinforcement learning," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 594–608, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3381646.

[9] A. Rodríguez, A. Navarro, K. Nikov, J. Nunez-Yanez, R. Gran, D. Suárez Gracia, and R. Asenjo, "Lightweight asynchronous scheduling in heterogeneous reconfigurable systems," *J. Syst. Archit.*, vol. 124, Art. no. 102398, 2022, doi: 10.1016/j.sysarc.2022.102398.

[10] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, "Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures," *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024, doi: 10.1016/j.future.2023.08.022.

[11] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, "StarPU: A unified platform for task scheduling on heterogeneous multicore architectures," *Concurrency Comput.: Pract. Exper.*, 2011 (journal version; see also Euro-Par 2009, LNCS 5704, pp. 863–874).